

Checkpoint 3 Summary: Attacks + Calibration Baseline

AI Explanation Reliability Under Attack

1. Checkpoint 3 Goal

Implement the adversarial attacks, measure how often they fool the classifier, and fix the threshold that will later decide whether an attacked image's explanation counts as deceptive. The threshold had to be fixed before any attacked image's explanation was examined, so the calibration step was run and committed first.

What the project measures: the outcome is explanation similarity, meaning how similar the Grad-CAM heatmap of an attacked image is to the heatmap of its clean original (SSIM). It does not measure whether a person finds an explanation plausible, or whether Grad-CAM faithfully reflects the model's reasoning. There is no user study, so this project draws no conclusions about human trust.

2. Work Completed

- Implemented `src/attacks.py`: FGSM and PGD (torchattacks) at epsilon 0.01, 0.03 and 0.05, six configurations in total.
- Evaluated all six on a 9,000-image evaluation split and recorded attack success rate and adversarial accuracy.
- Implemented `src/calibration.py`: a seeded split of the test set, a noise-floor distribution of Grad-CAM SSIM, and the locked threshold.
- Implemented `src/metrics.py` (SSIM) and batched Grad-CAM in `src/gradcam_utils.py`.
- Saved training logs (`results/logs/`), pinned exact library versions, and wrote a README documenting every parameter.

3. Methods and Parameters

Item	Setting
Classifier	ResNet-18 (CIFAR stem), checkpoint from epoch 28, test accuracy 91.03%
Data split	Seeded random split (seed 0) of the 10,000 test images: 1,000 calibration pool, 9,000 evaluation. The two never overlap.
Pixel scale	Attacks and noise act on [0, 1] pixels; normalization is applied inside the model
FGSM	Untargeted; one step of size epsilon along the sign of the cross-entropy loss gradient; clipped to [0, 1]
PGD	Untargeted, L-infinity; 10 steps; step size $2.5 \times \text{epsilon} / 10$; random start inside the epsilon ball; cross-entropy loss; projected to the epsilon ball and [0, 1]
Attack run	True labels, eval mode, batch size 250, <code>torch.manual_seed(0)</code> before each configuration; the run aborts if any perturbation exceeds epsilon
Grad-CAM	<code>pytorch-grad-cam 1.5.7</code> on <code>layer4[-1]</code> (native 4x4 map). Gradients averaged per channel, weighted sum of activations, ReLU, min-max scaled to [0, 1] per image, bilinearly resized to 32x32. Explains the model's own top-1 prediction.
SSIM	<code>skimage structural_similarity</code> , <code>data_range 1.0</code> , default 7x7 window, between two 32x32 heatmaps
Calibration noise	Random +/-0.01 sign noise on every pixel channel (same per-pixel size as an FGSM step at epsilon 0.01, random direction), clipped to [0, 1]; seed 0
Threshold rule	25th percentile of the noise-floor SSIM distribution. An attacked example is called deceptive if its SSIM to the clean heatmap is at or above the threshold.

4. Results

4.1 Attack success (evaluation split)

The classifier was correct on 8,198 of the 9,000 evaluation images before attack (91.09%). Success rate is the share of those 8,198 that become misclassified; adversarial accuracy is accuracy over all 9,000 attacked images.

Attack	Epsilon	Success rate	Adversarial accuracy	Images attacked
FGSM	0.01	77.6%	20.4%	6,362
FGSM	0.03	91.2%	8.0%	7,479
FGSM	0.05	91.9%	7.5%	7,531
PGD	0.01	97.7%	2.1%	8,011
PGD	0.03	100%	0.0%	8,198
PGD	0.05	100%	0.0%	8,198

PGD is stronger than FGSM at every epsilon, as expected for an iterative attack. FGSM's success barely rises from epsilon 0.03 to 0.05, a known limit of a single large step. Every configuration has far more than the 500 successfully attacked images the plan requires, and the largest perturbation observed in each run equalled its epsilon to floating-point precision.

4.2 Noise-floor calibration and the locked threshold

905 of the 1,000 calibration images were classified correctly and used. Each was paired with a copy carrying random ± 0.01 noise, and the SSIM between their Grad-CAM heatmaps was computed. Random noise changed the predicted class for 13 of the 905 images; those pairs were kept.

Statistic	Value
Pairs	905
SSIM mean / median	0.969 / 0.994
SSIM minimum / maximum	-0.176 / 0.9999
Percentile rule	25th percentile
Locked threshold	SSIM ≥ 0.9802

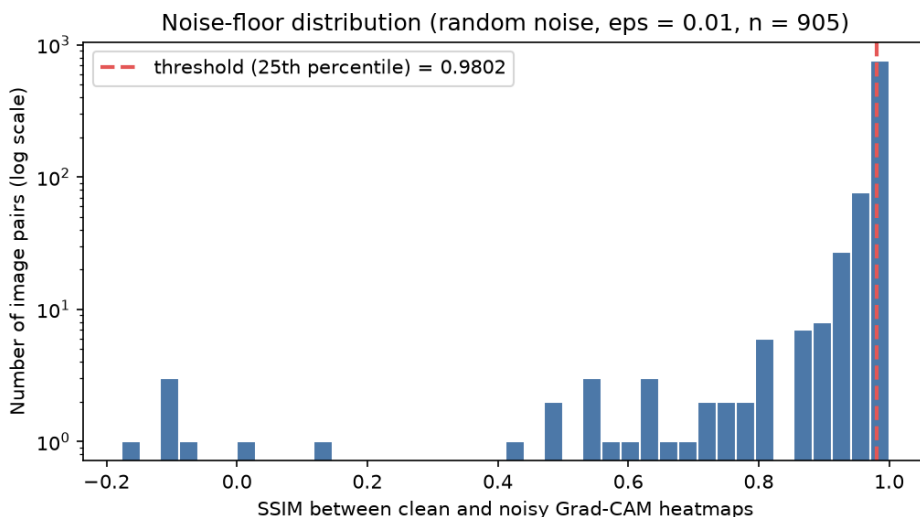


Figure 1. Noise-floor SSIM distribution (log scale) with the locked threshold.

Almost every pair sits very close to 1.0, so random noise barely changes the heatmaps. That makes the threshold strict: an attacked image's explanation must be nearly identical to the clean one to count as deceptive. I did not adjust the percentile or the threshold after seeing this; a low deceptive rate in later checkpoints would be a valid finding, not a reason to change the rule.

5. Reproducibility

Repository (private): github.com/Davidfaley360/explanation-reliability-under-attack

Key commits, in the order they were made:

1a45546 2026-10-06 Lock deceptive-explanation threshold from noise-floor calibration

aaf0ddc 2026-10-06 Add FGSM/PGD attack evaluation, training logs, seeded logging, pinned deps

7dbf768 2026-10-06 Add attack results table and README with full experimental parameters

The threshold commit precedes the attack code and the attack results, so the threshold could not have been tuned to them.

Environment: Apple M2 MacBook Air, macOS, Python 3.13.1, torch 2.14.0, torchvision 0.29.0, torchattacks 3.5.1, grad-cam 1.5.7, scikit-image 0.26.0; all versions pinned in requirements.txt. Seeds: split 0, calibration noise 0, attacks 0. Training logs are in results/logs/. The deployed checkpoint came from a training run made before a --seed option existed, so it is not bit-for-bit reproducible; the log records this.

Commands (in order): python -m src.train --epochs 30 --device mps; python -m src.calibration --device mps; python -m src.attacks --device mps.

6. Challenges and How They Were Resolved

- Slow, hanging training. Early runs took hours per epoch. The cause was the project folder sitting in an iCloud-synced Desktop folder, which made file access stall. Moving the project and environment to a normal folder cut import time from hours to about one second.
- Attack perturbations larger than allowed. A first smoke test showed changes about four times the epsilon budget. A torchattacks normalization option had been used incorrectly. The fix was to wrap the model so it accepts [0, 1] pixels, and a permanent check now stops the run if any perturbation exceeds epsilon. The invalid numbers were discarded.
- Keeping the threshold honest. The plan requires fixing it before attack results exist. The calibration script writes the threshold once and refuses to overwrite it, and it was committed before the attack code was written.
- Design choices the plan left open (noise type, size of the held-out set, PGD step settings) were made explicitly and recorded in the README so they can be reviewed.

7. Limitations

- Heatmap similarity is not human-perceived plausibility and not explanation faithfulness; the word deceptive refers only to the SSIM rule above.
- The Grad-CAM map is natively 4x4 and upsampled to 32x32, so heatmaps are coarse and smooth, which affects how sensitive SSIM is.
- One dataset, one model, two attacks and three strengths; conclusions apply to this controlled setting only.
- The checkpoint epoch was chosen using test accuracy on the same test set that supplies the calibration and evaluation images, so reported accuracy is slightly optimistic.
- The threshold depends on the chosen noise model (random sign noise at 0.01).

8. Timeline and Next Steps

Checkpoint	Status	Completed
1. Foundation	Done	Sep 4, rerun Sep 9
2. Full training + Grad-CAM	Done	Sep 27
3. Attacks + calibration	Done	Oct 6
4. Attack, explain, compare pipeline (subset)	Next	
5. Full-scale results and analysis	Planned	

6. Final report and video	Planned	
---------------------------	---------	--

Checkpoint 4 next steps:

- Add IoU (top 20% of pixels) and correlation as supporting metrics in src/metrics.py.
- Build src/pipeline.py: for each successfully attacked image, compute the clean and attacked Grad-CAM heatmaps, their SSIM, IoU and correlation, and apply the fixed 0.9802 threshold.
- Run it on a 500 to 1,000 image subset per configuration and save side-by-side before/after examples, including any deceptive cases.